

Reproducible Operations: NixOS and the Build-Service Trust Boundary

The model: declarative configuration, generations, rollback I

NixOS expresses an entire system — packages, services, users, kernel settings — as a declarative configuration evaluated by the Nix package manager, and activates changes as numbered *generations* (community 2026e). Activation builds a new system closure, atomically switches to it, and leaves prior generations bootable, so `nixos-rebuild` can roll the system configuration back to an earlier point (community 2026f). Nix builds run in isolated filesystem and process namespaces, making controlled build inputs a starting point for reproducible construction, though the reproducibility project documents remaining sources of nondeterminism (N. Project 2026g, 2026j).

The model: declarative configuration, generations, rollback I

The package manager itself has been evolving on a cadence that matters for security analysis. Nix 2.34 (released early 2026) tightened sandbox behavior — blocking extended-attribute propagation inside builds — and extended build tracing, and Nix 2.35.0 (June 22, 2026) continues that line; the 2.35 release notes also fold secrets handling into the manual as a first-class chapter, recognizing that credential delivery into builds and services is a security surface rather than an afterthought (N. Project 2026f). Version discipline is part of the update-operations property: an installation's security posture is a function of which Nix release it actually runs, and the advisory record below shows the Nix releases themselves requiring prompt updates.

The model: declarative configuration, generations, rollback I

In this review's threat model, NixOS's value is not confinement. It is the ability to make intended system state explicit, auditable, and replaceable — to construct, update, audit, and replace tightly scoped environments that are separately isolated. A deployment whose configuration is a reviewed, versioned artifact can answer “what exactly is running here?” in a way an imperatively mutated machine cannot, which is the foundation the authority architecture in sec. 10 builds on when an agent proposes configuration changes.

Rollback is not recovery I

Generation rollback switches *system configuration* generations; it does not restore mutable state (community 2026f). The community discussion of rolling back data as well as configuration documents the mismatch that arises when a service's database has migrated but its software is rolled back (community 2026j). Three consequences follow for the persistence-and-recovery property.

First, **rollback is not a malware-remediation plan**. A compromised agent's files in `/home`, stolen credentials, exfiltrated data, and accepted remote changes survive a generation switch; the state a rebuild can restore is the state Nix owns. Second, **older retained generations are potentially vulnerable software, not trusted recovery images**: they are pinned to the package versions of their era, including any advisories published since — a concern the 2026 Nix advisories make concrete, since a generation built with an

unpatched Nix release carries the advisory's exposure until the manager itself is updated. Third, **data recovery is a separate discipline** — independent backups and rehearsed restoration (sec. 12) — that declarative configuration does not substitute for. The review's recovery scenario in sec. 16 therefore requires distinguishing deletion of an environment from revocation of any credentials it could have reached.

Reproducible deployment versus verified reproducible builds I

The most misused word in this space is “reproducible,” and NixOS is where the confusion does real security harm. Two distinct claims must be kept apart (N. Project 2026j):

Reproducible deployment versus verified reproducible builds I

- ▶ **Reproducible deployment:** a system can be recreated from its declarative configuration — the same expression yields a coherent, working system. NixOS provides this well (community 2026e).
- ▶ **Verified reproducible builds:** every package has been independently rebuilt and compared bit-for-bit against the binary actually deployed, proving the artifact corresponds to its claimed inputs.

Reproducible deployment versus verified reproducible builds I

The first does not imply the second. The Nix sandbox's dependency specification and filesystem/process isolation alone do not guarantee reproducible outputs (N. Project 2026g, 2026j), and the reproducibility project exists precisely because the gap is being closed package by package, not closed by declaration. The 2026 measurements quantify how far the gap has closed and where it remains: the NixOS ISO closure (channel `nixos.iso_gnome`) reaches 95.18% reproducibility across its full paths (N. Project 2026h), while a build-closure analysis reports 99.48% of packages rebuilding bit-for-bit (N. Project 2026h); a large-scale rebuild study of Nixpkgs documents the remaining nondeterminism classes and their distribution across the package set (Malka et al. 2025). These are measured rates for specific artifacts, not a general guarantee — the same discipline that keeps this review from scoring platforms applies to interpreting them.

Reproducible deployment versus verified reproducible builds I

Neither form of reproducibility, finally, establishes that the source, configuration, or upstream dependency is *safe* — reproducibility answers “is this the artifact that source produces?”, not “should anyone run this?”. This mirrors the review’s broader separations: authenticity from provenance, rollback from recovery, and containment from authority (sec. 4).

Six misconceptions that matter against offensive agents I

Because Nix's vocabulary sounds stronger than its documented guarantees, the source assessment's misconception list is worth keeping as a standing table. tbl. 1 states each with the documented fact and its consequence — updated where the 2026 record changed the facts.

Six misconceptions that matter against offensive agents I

Table 1: Six NixOS misconceptions, the documented facts that correct them, and the consequences against offensive agents.

Misconception	Documented fact	Consequence
"The Nix sandbox confines everything I run."	The documented sandbox concerns builds, including specified exceptions such as network access for fixed-output derivations (N. Project 2026g).	The sandbox is a build-time mechanism, not a runtime security boundary around an AI agent or programs installed through Nix.
"A signed cache artifact is proven to match reviewed source."	Cache signatures distinguish trusting a signer from proving an output was produced by the expected derivation (community 2026m).	Authenticity is valuable, but a trusted supplier can still distribute a bad artifact; signatures do not establish provenance — which is why the provenance tooling below matters.
"Putting secrets into declarative configuration is harmless."	The Nix manual treats secret management as a first-class concern: the store is world-readable and secrets can reach external caches, so secrets belong in runtime-protected stores — now with dedicated manual guidance in the 2.35 documentation (N. Project 2026k, 2026f).	Configuration cleanliness must not become credential exposure; an agent with store access can read declaratively embedded secrets.
"NixOS already has a fully integrated mandatory-access-control baseline."	The wiki documents incomplete SELinux integration and, as of April 2026, incomplete AppArmor integration (community 2026k).	Confinement is possible but not presumed; do not assume a comprehensive default policy exists.
"Secure Boot is a settled, integrated default."	Lanzaboote remains outside nixpkgs — its packaging PR is still open (community 2026b) — and its repository documents that key management is outside the project's full scope; measured-boot (TPM PCR) support landed in April 2026, and the bootspec removal in nixpkgs broke v1.0.0 until v1.1.0 fixed the integration (community 2026a, 2026c).	A carefully configured installation and a stock default are different claims; boot-integrity stance is partial pending upstream integration — and the version pinning is load-bearing.

Six misconceptions that matter against offensive agents II

"The hardened profile is a safe universal upgrade."

The hardened profiles were **removed** from nixpkgs on March 22, 2026 (PR nixpkgs#501199), with the 26.05 release notes listing the removal as a backward incompatibility; the maintainer discussion that preceded it argued the profile lacked a coherent baseline and could undermine browser sandboxing, and the NixOS Hardening wiki now carries the composable guidance instead (community 2026h, 2026d, 2026g).

Hardening menus are not defaults; the ecosystem itself concluded that an unmaintained blanket profile is worse than documented per-concern guidance — the defaults-over-menus forecast in sec. 15.

Six misconceptions that matter against offensive agents I

For agent-bearing deployments the first and third rows bind hardest: an autonomous agent executing on a NixOS host is *not* sandboxed by Nix, and any secret that reached declarative configuration is readable by whatever that agent's processes can read.

The package manager is part of the trusted computing base I

The 2026 advisory chain makes the trusted-computing-base argument with unusual clarity, because the advisories landed in sequence against the same component. On April 7, 2026, Nix published a security advisory describing a symlink-following flaw during fixed-output derivation registration that could allow users permitted to submit builds to gain root in multi-user installations, including with sandboxed Linux builds enabled; the advisory lists fixes across version branches, including 2.34.5 and 2.33.4 (N. Project 2026b). Later in the year two further advisories followed: one in the NAR archive parser, where unbounded recursion — now bounded — allowed resource exhaustion during store-path ingestion at high severity (CVSS 7.5) (N. Project 2026d); and one enabling path traversal on `--unpack` extraction, an

The package manager is part of the trusted computing base I

attacker-controlled path escape out of the intended destination (CVSS 4.3) (N. Project 2026c). A third issue, a TOCTOU race in recursive-Nix builds, required the 2.35.0 fix to close (N. Project 2026a, 2026f). Each advisory is fixed in current releases; the structural observation is what matters.

The finding is not evidence that Nix is uniquely unsafe, nor that the flaws remain unpatched. It is evidence for a structural claim: **the build service is part of the trusted computing base**. A component with root authority over the machine, whose input includes build submissions and attacker-named artifacts (NAR archives, tarballs, path names), is a privileged parser of attacker-influenced content — the same shape as the QSB-118 dom0 injection in sec. 5, and the same lesson: prompt patching of the management layer is not optional hygiene but the update-operations property of sec. 4 doing real security work. The chain

The package manager is part of the trusted computing base I

also illustrates the second-order risk of pinned environments: a deployment frozen on an older Nix release accumulates advisory exposure in the very tool that delivers its other updates.

The deployment consequence follows directly from the threat model in sec. 3: an agent allowed to submit attacker-controlled builds should not share a trust domain with the owner's most valuable credentials merely because the builds use Nix. In the trust-domain architecture of sec. 10, build submission belongs with agent execution (disposable, scoped), not with administration or the credential service — and a multi-user Nix installation hosting untrusted build submitters inherits a root-equivalent trust requirement that its operators must patch and monitor (N. Project 2026b, 2026d). The Nixpkgs security tracker (N. Project 2026i) coordinates vulnerability matching and mitigation, which supports the update-operations stance but does not by itself deliver

The package manager is part of the trusted computing base I

patch latency.

Provenance: from signatures to verification I

The supply-chain-trust property asks who may introduce or approve executable code, and 2026 hardened the tooling that answers it. Nixpkgs now requires `meta.sourceProvenance` on packages — an explicit, machine-readable declaration of whether a build artifact derives from source, from an upstream binary blob, or from an unfree distribution — turning provenance from documentation into an enforced field of every package definition (community 2026i). Verification tooling closes the loop on the binary side: sbomnix generates SBOMs (CycloneDX/SPDX) for a Nix closure and can attach SLSA provenance attestations ((TII) 2026); Trustix compares independently rebuilt derivations at scale, exposing bit-for-bit divergence between builders (community 2026l); and Determinate's `nix provenance show/verify` commands

Provenance: from signatures to verification I

bring attestation checking into the package-manager CLI itself. None of this converts a signature into proof of benignity — the misconception table's second row still stands — but together they make the provenance half of the question answerable by tooling rather than trust, which is exactly the direction the supply-chain-trust property demands for agent-bearing deployments: an auditor can now ask “what does this closure claim, and what was independently rebuilt?” as data.

Update operations: the support-clock discipline I

NixOS 26.05's announcement specifies seven months of bug fixes and security updates, ending December 31, 2026 (N. Project 2026e). A stable configuration revision is a point-in-time choice, not a permanently safe one: after support ends, an unchanged deployment accumulates unpatched vulnerabilities while its configuration remains byte-identical. The update-operations property therefore binds declarative systems as hard as any other: establish an explicit channel-update and rebuild cadence, treat held-back generations as aging software, and fold the tracker into the operator's patch process (N. Project 2026i, 2026e). This is where Nix's strengths compound — because the system state is declarative, an update is a *reviewable configuration change* rather than an opaque mutation, and a failed rebuild leaves the prior generation bootable

Update operations: the support-clock discipline I

(community 2026f). The 2026 advisory chain gives the discipline a concrete rhythm: each Nix release that closes an advisory is also a reminder that the update path itself is the artifact being protected.

Composition with Qubes I

Nix and Qubes are not competing answers: one principally organizes runtime trust domains, the other organizes how software environments are constructed. The natural composition uses Nix to construct the contents of compartmentalized qubes — declarative, reviewable, replaceable environments inside a containment architecture. The Qubes NixOS-template issue documents community work toward exactly this (Q. O. Project 2026a), but the integration is not a product: Qubes warns that community templates do not receive updates from the Qubes project itself (Q. O. Project 2026b), so a hand-built Qubes-plus-NixOS stack inherits a maintenance obligation for template security updates, boot support, and desktop integration that its owner must explicitly own (community 2026k, 2026b). Combining two attractive designs does not automatically improve practical security; the composition must be operated, patched,

and reviewed as its own deployment.

NixOS is especially attractive for a capable operator building auditable, replaceable development or server environments. On the candidate–property matrix its update-operations and supply-chain-trust stances are strong — declarative state, channel discipline, the security tracker, and now enforced provenance metadata and verification tooling are documented and coherent (community 2026i; (TII) 2026) — while its application-confinement stance is weak pending integrated MAC and boot-integrity baselines (community 2026k, 2026a). It is not the default recommendation over Qubes for containing a hostile desktop workload, and it is not sufficient by itself to make a powerful autonomous coding agent safe: the sandbox that defines its build discipline is build-time, its secrets model demands deliberate handling — now with first-class manual

guidance (N. Project 2026f) — and its build service is root-authority code with a 2026 advisory chain that must be treated as trusted base (N. Project 2026g, 2026k, 2026b, 2026d, 2026c). The strongest arrangement — and the one this review's architecture chapters target — uses Nix's declarative construction *inside* a separately enforced, least-privilege execution architecture, where generation discipline supplies replaceability and the containment layer supplies what Nix deliberately does not.